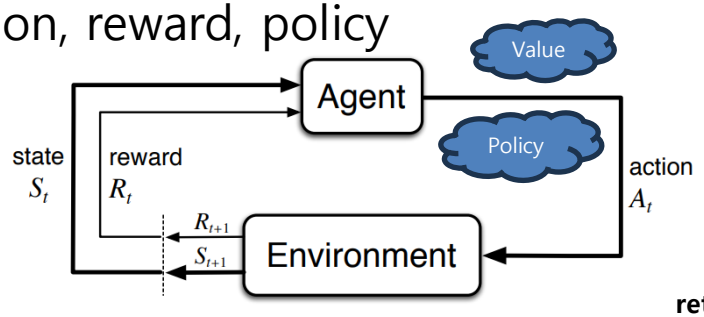
# Dynamic Programming

Prof. Tae-Hyoung Park
Dept. of Intelligent Systems & Robotics, CBNU

# MDP Summary

- State, action, reward, policy




**return** $G_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \cdots$

- Value
  - State-value: $v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]$, Action-value: $q_\pi(s,a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]$
  - Bellman equation

$$v_\pi(s) = \sum_{a,s'} \pi(a|s)\, p(s'|s,a)\, \{r(s,a,s') + \gamma v_\pi(s')\} \qquad q_\pi(s,a) = \sum_{s'} p(s' \mid s,a)\left\{r(s,a,s') + \gamma \sum_{a'} \pi(a' \mid s') q_\pi(s',a')\right\}$$

- Optimal Value
  - State-value: $v_*(s) = \max_\pi v_\pi(s)$, action-value: $q_*(s,a) = \max_\pi q_\pi(s,a)$
  - Bellman optimality equation

$$v_*(s) = \max_a \sum_{s'} p(s' \mid s,a)\{r(s,a,s') + \gamma v_*(s')\} \qquad q_*(s,a) = \sum_{s'} p(s'|s,a)\left\{r(s,a,s') + \gamma \max_{a'} q_*(s',a')\right\}$$

  - Optimal policy (deterministic policy case)

$$\pi_*(a|s) = \begin{cases} 1, & \text{if } a = \arg\max_{a \in \mathcal{A}} q_*(s,a) \\ 0, & \text{otherwise.} \end{cases} \qquad \mu_*(s) = \arg\max_a q_*(s,a) \quad \text{(optimal action)}$$

# Policy Evaluation & Improvement

- Policy Evaluation (평가)
  - 현재의 policy $\pi(a|s)$ 가 얼마나 우수한지 평가

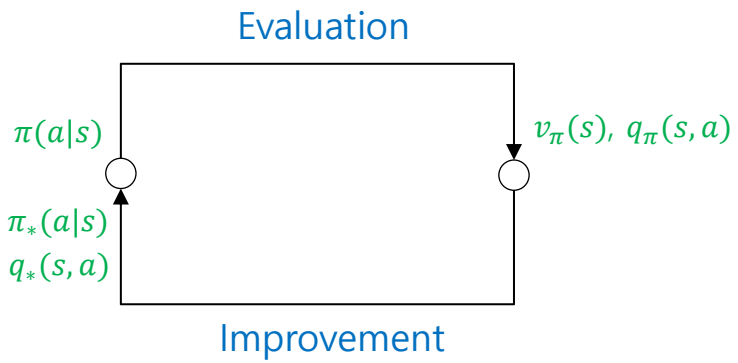
  ⇒ Calculate the action-value $q_\pi(s,a)$ and state-value $v_\pi(s)$

  <Bellman Equation>

- Polish Improvement (개선)
  - 현재의 policy 를 새로운 policy 로 갱신

  ⇒ Calculate $q_*(s,a)$ and obtain optimal policy $\pi_*(a|s)$

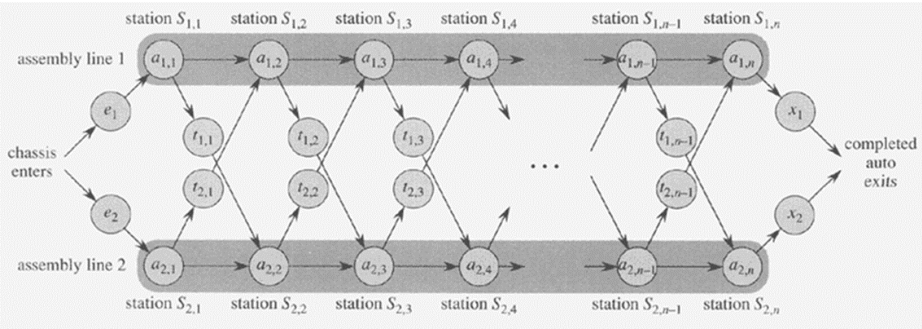
  <Bellman Optimality Equation>



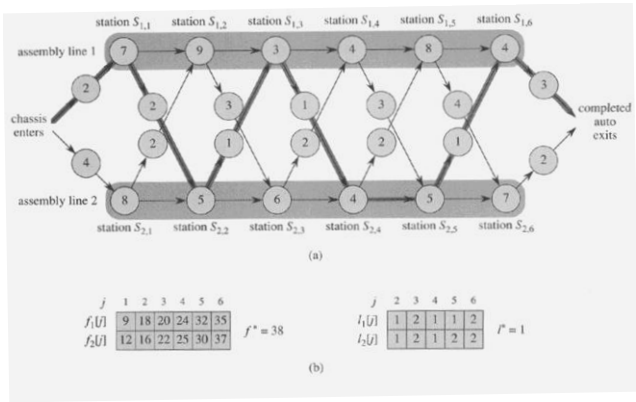

☞ How to solve ?

# Dynamic Programming

- DP
  - Solving a complex problem by breaking it down into simpler subproblems
  - Storing the results of those subproblems so you don't have to re-calculate them
  - ➢ Curse of dimensionality: computation increases significantly according to dim.
- Example: Assembly scheduling problem




$$f_1[1] = e_1 + a_{1,1}$$
$$f_2[1] = e_2 + a_{2,1}$$
$$f_1[2] = \min(f_1[1] + a_{1,2}, f_2[1] + t_{2,1} + a_{1,2})$$
$$f_2[2] = \min(f_2[1] + a_{2,2}, f_1[1] + t_{1,2} + a_{2,2})$$
$$\vdots$$
$$f_1[j] = \min(f_1[j-1] + a_{1,j}, f_2[j-1] + t_{2,j-1} + a_{1,j})$$
$$f_2[j] = \min(f_2[j-1] + a_{2,j}, f_1[j-1] + t_{1,j-1} + a_{2,j})$$
$$\vdots$$
$$f^* = \min(f_1[n] + x_1, f_2[n] + x_2)$$



# Dynamic Programming

- Example 2: Longest common subsequence

$$c[i,j] = \begin{cases} 0 & i = 0 \; or \; j = 0 \\ c[i-1, j-1] + 1 & i, j > 0 \; and \; x_i = y_j \\ \max(c[i, j-1], c[i-1, j]) & i, j > o \; and \; x_i \neq y_j \end{cases}$$

| $i$ | $j$ | 0 | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|---|---|
| | | $y_j$ | $B$ | $D$ | $C$ | $A$ | $B$ | $A$ |
| 0 | $x_i$ | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 1 | $A$ | 0 | ↑0 | ↑0 | ↑0 | ↖1 | ←1 | ↖1 |
| 2 | $B$ | 0 | ↖1 | ←1 | ←1 | ↑1 | ↖2 | ←2 |
| 3 | $C$ | 0 | ↑1 | ↑1 | ↖2 | ←2 | ↑2 | ↑2 |
| 4 | $B$ | 0 | ↖1 | ↑1 | ↑2 | ↑2 | ↖3 | ←3 |
| 5 | $D$ | 0 | ↑1 | ↖2 | ↑2 | ↑2 | ↑3 | ↑3 |
| 6 | $A$ | 0 | ↑1 | ↑2 | ↑2 | ↖3 | ↑3 | ↖4 |
| 7 | $B$ | 0 | ↖1 | ↑2 | ↑2 | ↑3 | ↖4 | ↑4 |

# Iterative Bellman Equation

- Bellman Equation
  - Definition

$$v_\pi(s) \equiv \mathbb{E}_\pi[G_t|S_t = s] = \mathbb{E}_\pi[R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \cdots |S_t = s]$$

  - Bellman equation

$$v_\pi(s) = \sum_{a,s'} \pi(a \mid s)\, p(s' \mid s,a)\{r(s,a,s') + \gamma v_\pi(s')\}$$

    - '다음 상태'의 가치함수 $v_\pi(s')$ 와 '현재 상태' 의 가치함수 $v_\pi(s)$ 의 관계식
    - 연립 방정식 문제 => 복잡한 문제에 적용 어려움

- Iterative Bellman equation

$$\boxed{V_{k+1}(s) = \sum_{a,s'} \pi(a \mid s)\, p(s' \mid s,a)\{r(s,a,s') + \gamma V_k(s')\}}$$

$$V_0(s) \rightarrow V_1(s) \rightarrow \ldots \rightarrow V_k(s) \rightarrow V_{k+1}(s) \rightarrow \ldots \approx v_\pi(s)$$ (dynamic programming)

    - $V_k(s)$: $k$ 번째로 갱신(update) 된 state-value 의 '추정치' (실제 가치 $v_\pi(s)$ 와 다름!!)
    - 추정치 $V_k(s')$ 로 부터 추정치 $V_{k+1}(s)$ 를 개선 (improvement): bootstrapping

☞ 연립방정식 문제를 dynamic programming 으로 전환하여 해결

# Iterative Bellman Equation

- Deterministic State-Transition Case
  - Deterministic state-transition probability

$$p(s'|s,a) = \begin{cases} 1 & ,if\ s' = f(s,a) \\ 0 & if\ s' \neq f(s,a) \end{cases}$$

  - Iterative Bellman equation

$$V_{k+1}(s) = \sum_{a,s'} \pi(a \mid s)\, p(s' \mid s,a)\{r(s,a,s') + \gamma V_k(s')\}$$

$$\rightarrow \quad V_{k+1}(s) = \sum_{a} \pi(a|s) \sum_{s'} p(s'|s,a)\{r(s,a,s') + \gamma V_k(s')\}$$

→ For the deterministic state-transition

$$\boxed{V_{k+1}(s) = \sum_{a} \pi(a \mid s)\{r(s,a,s') + \gamma V_k(s')\}} \qquad s' = f(s,a)$$

# Policy Evaluation

- Iterative Policy Evaluation
  - 목적: calculate $v_\pi(s)$
  - 방법: $V_0(s) \to V_1(s) \to \ldots \to V_k(s) \to V_{k+1}(s) \to \ldots \approx v_\pi(s)$
  - 알고리즘

Input $\pi$, the policy to be evaluated
Initialize an array $V(s) = 0$, for all $s \in \mathcal{S}^+$
Repeat
  $\Delta \leftarrow 0$
  For each $s \in \mathcal{S}$:
    $v \leftarrow V(s)$
    $V(s) \leftarrow \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a)\left[r + \gamma V(s')\right]$
    $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
until $\Delta < \theta$ (a small positive number)
Output $V \approx v_\pi$

$\mathcal{S}^+ = \mathcal{S} \cup \{\text{terminal state}\}$

(Ex) $V_k$

$k = 0$

| | | | |
|---|---|---|---|
| 0.0 | 0.0 | 0.0 | 0.0 |
| 0.0 | 0.0 | 0.0 | 0.0 |
| 0.0 | 0.0 | 0.0 | 0.0 |
| 0.0 | 0.0 | 0.0 | 0.0 |

$k = 1$

| | | | |
|---|---|---|---|
| 0.0 | -1.0 | -1.0 | -1.0 |
| -1.0 | -1.0 | -1.0 | -1.0 |
| -1.0 | -1.0 | -1.0 | -1.0 |
| -1.0 | -1.0 | -1.0 | 0.0 |

$k = 2$

| | | | |
|---|---|---|---|
| 0.0 | -1.7 | -2.0 | -2.0 |
| -1.7 | -2.0 | -2.0 | -2.0 |
| -2.0 | -2.0 | -2.0 | -1.7 |
| -2.0 | -2.0 | -1.7 | 0.0 |

$k = 3$

| | | | |
|---|---|---|---|
| 0.0 | -2.4 | -2.9 | -3.0 |
| -2.4 | -2.9 | -3.0 | -2.9 |
| -2.9 | -3.0 | -2.9 | -2.4 |
| -3.0 | -2.9 | -2.4 | 0.0 |

$k = 10$

| | | | |
|---|---|---|---|
| 0.0 | -6.1 | -8.4 | -9.0 |
| -6.1 | -7.7 | -8.4 | -8.4 |
| -8.4 | -8.4 | -7.7 | -6.1 |
| -9.0 | -8.4 | -6.1 | 0.0 |

$k = \infty$

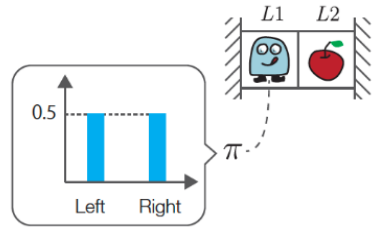
| | | | |
|---|---|---|---|
| 0.0 | -14. | -20. | -22. |
| -14. | -18. | -20. | -20. |
| -20. | -20. | -18. | -14. |
| -22. | -20. | -14. | 0.0 |

# Policy Evaluation

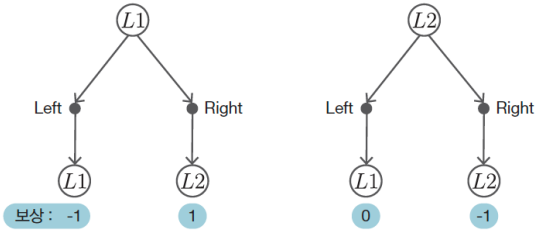
- (Example) 2-Grid World
  - $V_0(s)$

    $V_0(L1) = 0$

    $V_0(L2) = 0$




L1에서 L2로 이동하면 +1 보상, 벽에 부딪히면 −1 보상)




  - $V_1(s)$

$$V_{k+1}(s) = \sum_a \pi(a \mid s)\{r(s,a,s') + \gamma V_k(s')\}$$

$V_1(L1) = \pi(LEFT|L1)\{r(L1, LEFT, L1) + \gamma V_0(L1)\} + \pi(RIGHT|L1)\{r(L1, RIGHT, L2) + \gamma V_0(L2)\}$

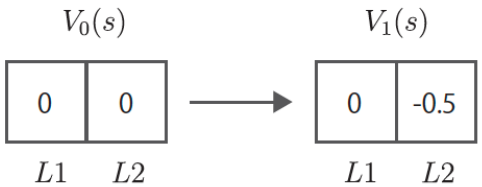
= 0.5 {-1 + 0.9 $V_0(L1)$} + 0.5 {1 + 0.9 $V_0(L2)$}

= 0

$V_1(L2) = \pi(LEFT|L2)\{r(L2, LEFT, L1) + \gamma V_0(L1)\} + \pi(RIGHT|L2)\{r(L2, RIGHT, L2) + \gamma V_0(L2)\}$

= 0.5 {0 + 0.9 $V_0(L1)$} + 0.5 {-1 + 0.9 $V_0(L2)$}

= -0.5

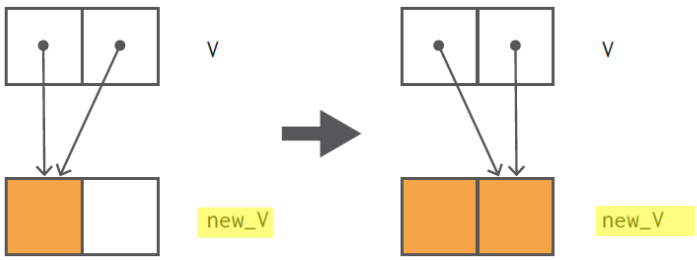


# Policy Evaluation

- (Example) 2-Grid World
  - Program

```
V = {'L1': 0.0, 'L2': 0.0}
new_V = V.copy()

cnt = 0  # 갱신 횟수 기록
while True:
    new_V['L1'] = 0.5 * (-1 + 0.9 * V['L1']) + 0.5 * (1 + 0.9 * V['L2'])
    new_V['L2'] = 0.5 * (0 + 0.9 * V['L1']) + 0.5 * (-1 + 0.9 * V['L2'])

    # 갱신된 양의 최댓값
    delta = abs(new_V['L1'] - V['L1'])
    delta = max(delta, abs(new_V['L2'] - V['L2']))

    V = new_V.copy()

    cnt += 1
    if delta < 0.0001:  # 임곗값 = 0.0001
        print(V)
        print('갱신 횟수:', cnt)
        break
```

Dictionary data structure




  - 결과

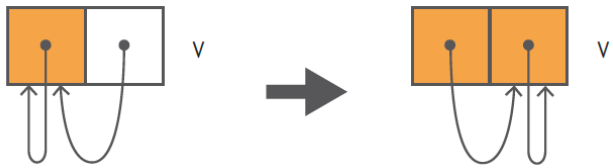
```
{'L1': -2.249167525908671, 'L2': -2.749167525908671}
갱신 횟수: 76
```

$$V_{76}(L1) = -2.249$$
$$V_{76}(L2) = -2.749$$

# Policy Evaluation

- Example: 2-Grid World
  - Program : In-place method

```
V = {'L1': 0.0, 'L2': 0.0}

cnt = 0
while True:
    t = 0.5 * (-1 + 0.9 * V['L1']) + 0.5 * (1 + 0.9 * V['L2'])
    delta = abs(t - V['L1'])
    V['L1'] = t

    t = 0.5 * (0 + 0.9 * V['L1']) + 0.5 * (-1 + 0.9 * V['L2'])
    delta = max(delta, abs(t - V['L2']))
    V['L2'] = t

    cnt += 1
    if delta < 0.0001:
        print(V)
        print('갱신 횟수:', cnt)
        break
```



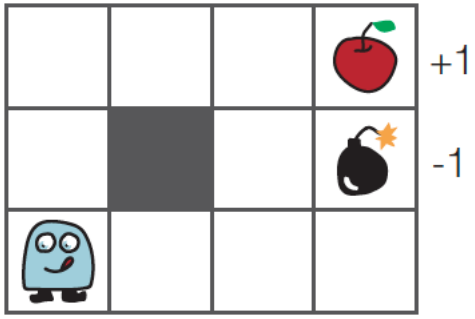

  - 결과

```
{'L1': -2.2493782177156936, 'L2': -2.7494201578106514}
갱신 횟수: 60
```

$V_{60}(L1) = -2.249$
$V_{60}(L2) = -2.749$

# GridWorld Class

- (Example) 3 x 4 Grid World

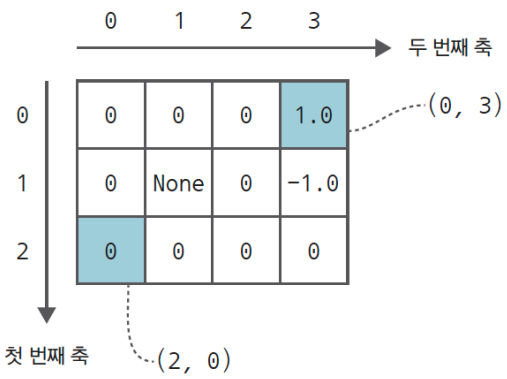



- 에이전트는 상하좌우 네 방향으로 이동할 수 있다.
- [그림 4-8]에서 회색 칸은 벽을 뜻하며 벽 안으로는 들어갈 수 없다.
- 그리드 바깥도 벽으로 둘러싸여 더 이상 나아갈 수 없다.
- 벽에 부딪히면 보상은 0이다.
- 사과는 보상 +1, 폭탄은 보상 −1, 그 외의 보상은 0이다.
- 환경의 상태 전이는 고유하다(결정적). 즉, 에이전트가 오른쪽으로 이동하는 행동을 선택하면 (벽만 없다면) 반드시 오른쪽으로 이동한다.
- 이번 문제는 일회성 과제로서, 사과를 얻으면 종료한다.

# GridWorld Class

- GridWorld Class
  - initialization

```python
import numpy as np

class GridWorld:
    def __init__(self):
        self.action_space = [0, 1, 2, 3]  # 행동 공간(가능한 행동들)
        self.action_meaning = {  # 행동의 의미
            0: 'UP',
            1: 'DOWN',
            2: 'LEFT',
            3: 'RIGHT',
        }

        self.reward_map = np.array(  # 보상 맵(각 좌표의 보상값)
            [[0, 0, 0, 1.0],
             [0, None, 0, -1.0],
             [0, 0, 0, 0]]
        )
        self.goal_state = (0, 3)    # 목표 상태(좌표)
        self.wall_state = (1, 1)    # 벽 상태(좌표)
        self.start_state = (2, 0)   # 시작 상태(좌표)
        self.agent_state = self.start_state   # 에이전트 초기 상태(좌표)
```



# GridWorld Class

- GridWorld Class
  - method

```python
class GridWorld:
    ...

    @property
    def height(self):
        return len(self.reward_map)

    @property
    def width(self):
        return len(self.reward_map[0])

    @property
    def shape(self):
        return self.reward_map.shape

    def actions(self):
        return self.action_space # [0, 1, 2, 3]

    def states(self):
        for h in range(self.height):
            for w in range(self.width):
                yield (h, w)
```

```python
env = GridWorld()

# env.height() 대신 env.height로 사용 가능
print(env.height)  # [출력 결과] 3

print(env.width)   # [출력 결과] 4
print(env.shape)   # [출력 결과] (3, 4)

for action in env.actions():  # 모든 행동에 순차적으로 접근
    print(action)

print('===')

for state in env.states():  # 모든 상태에 순차적으로 접근
    print(state)
```

```
0
1
2
3
===
(0, 0)
(0, 1)
...
(2, 3)
```

# GridWorld Class

- GridWorld Class

```python
class GridWorld:
    ...

    def next_state(self, state, action):
        # ❶ 이동 위치 계산
        action_move_map = [(-1, 0), (1, 0), (0, -1), (0, 1)]
        move = action_move_map[action]
        next_state = (state[0] + move[0], state[1] + move[1])
        ny, nx = next_state

        # ❷ 이동한 위치가 그리드 월드의 테두리 밖이나 벽인가?
        if nx < 0 or nx >= self.width or ny < 0 or ny >= self.height:
            next_state = state
        elif next_state == self.wall_state:
            next_state = state

        return next_state  # ❸ 다음 상태 반환

    def reward(self, state, action, next_state):
        return self.reward_map[next_state]
```

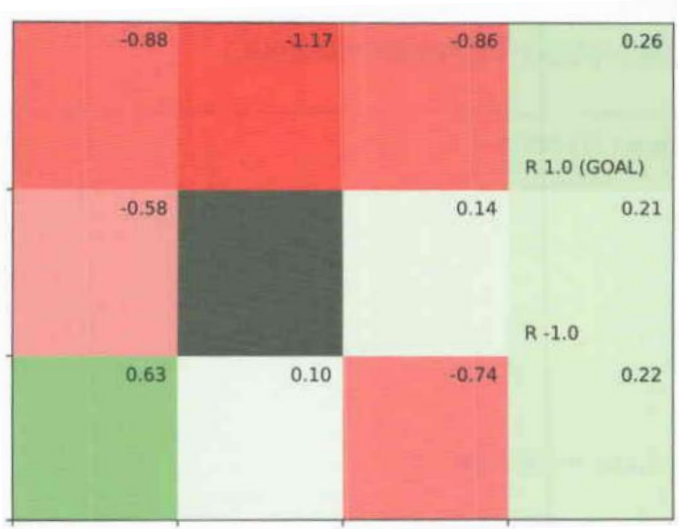
$s' = f(s, a)$

$r(s, a, s')$

# GridWorld Class

- GridWorld Class
  - 시각화 함수: render_v()

```
env = GridWorld()
V = {}
for state in env.states():
    V[state] = np.random.randn()  # 더미 상태 가치 함수
env.render_v(V)
```



# GridWorld Class

- defaultdict
  - 파이썬에서 제공하는 dictionary 자료구조
  - 자동 초기화 기능

```python
from common.gridworld import GridWorld

env = GridWorld()
V = {}

# 딕셔너리 원소 초기화
for state in env.states():
    V[state] = 0

state = (1, 2)
print(V[state])  # 상태 (1,2)의 가치 함수 출력
```

$V_k(s)$

```python
from collections import defaultdict  # defaultdict 임포트
from common.gridworld import GridWorld

env = GridWorld()
V = defaultdict(lambda: 0)

state = (1, 2)
print(V[state])  # [출력 결과] 0
```

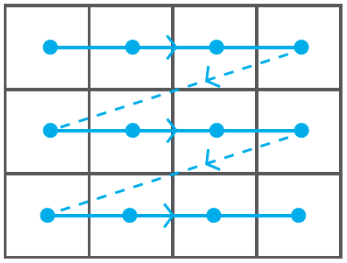
$\pi(a|s)$

```python
pi = defaultdict(lambda: {0: 0.25, 1: 0.25, 2: 0.25, 3: 0.25})

state = (0, 1)
print(pi[state])  # [출력 결과] {0:0.25, 1:0.25, 2:0.25, 3:0.25}
```

# Policy Evaluation

- One-Step Evaluation : $V_k \rightarrow V_{k+1}$

```python
def eval_onestep(pi, V, env, gamma=0.9):
    for state in env.states():  # ❶ 각 상태에 접근
        if state == env.goal_state:  # ❷ 목표 상태에서의 가치 함수는 항상 0
            V[state] = 0
            continue

        action_probs = pi[state]  # probs는 probabilities(확률)의 약자
        new_V = 0

        # ❸ 각 행동에 접근
        for action, action_prob in action_probs.items():
            next_state = env.next_state(state, action)
            r = env.reward(state, action, next_state)
            # ❹ 새로운 가치 함수
            new_V += action_prob * (r + gamma * V[next_state])

        V[state] = new_V
    return V
```



$a : \pi(a|s)$

{0: 0.25, 1: 0.25, 2: 0.25, 3: 0.25}

$$V_{k+1}(s) = \sum_a \pi(a \mid s)\{r(s,a,s') + \gamma V_k(s')\}$$

# Policy Evaluation

- Iterative Evaluation : $V_0 \rightarrow V_1 \rightarrow \cdots \approx v_\pi$
  - $V_k(s)$ 값이 수렴할 때 까지 one-step evaluation 을 반복

```python
def policy_eval(pi, V, env, gamma, threshold=0.001):
    while True:
        old_V = V.copy()  # 갱신 전 가치 함수
        V = eval_onestep(pi, V, env, gamma)

        # 갱신된 양의 최댓값 계산
        delta = 0
        for state in V.keys():
            t = abs(V[state] - old_V[state])
            if delta < t:
                delta = t

        # 임계값과 비교
        if delta < threshold:
            break
    return V
```

$V_k \rightarrow V_{k+1}$

$\Delta \leftarrow \max(\Delta, |v - V(s)|)$

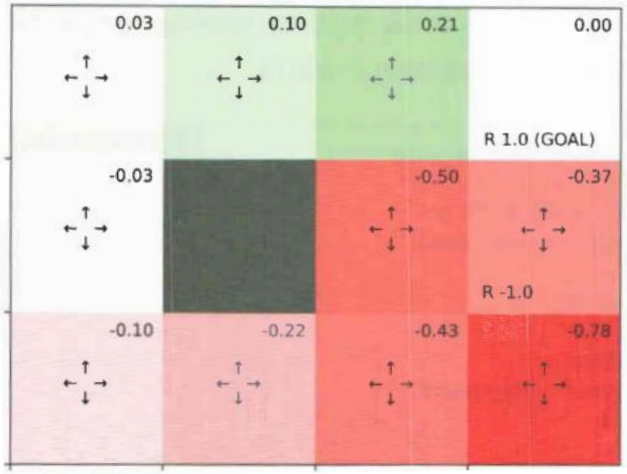
$\approx v_\pi(s)$

# Policy Evaluation

- Policy Evaluation : $v_\pi$
  - 주어진 policy $\pi$ 에 대한 state-value $v_\pi(s)\ \forall s$ 를 계산
  - Hyperparameter: $\gamma$ (discount rate)

```
env = GridWorld()
gamma = 0.9  # 할인율

pi = defaultdict(lambda: {0: 0.25, 1: 0.25, 2: 0.25, 3: 0.25})  # 정책
V = defaultdict(lambda: 0)  # 가치 함수

V = policy_eval(pi, V, env, gamma)  # 정책 평가
env.render_v(V, pi)  # 시각화
```

$v_\pi$



# 실습

## 실습 #1 gridworld.py

```python
import numpy as np
import common.gridworld_render as render_helper


class GridWorld:
    def __init__(self):
        self.action_space = [0, 1, 2, 3]  # 행동 공간(가능한 행동들)
        self.action_meaning = {  # 행동의 의미
            0: "UP",
            1: "DOWN",
            2: "LEFT",
            3: "RIGHT",
        }

        self.reward_map = np.array(  # 보상 맵(각 좌표의 보상 값)
            [[0, 0, 0, 1.0],
             [0, None, 0, -1.0],
             [0, 0, 0, 0]]
        )
        self.goal_state = (0, 3)    # 목표 상태(좌표)
        self.wall_state = (1, 1)    # 벽 상태(좌표)
        self.start_state = (2, 0)   # 시작 상태(좌표)
        self.agent_state = self.start_state   # 에이전트 초기 상태(좌표)

    @property
    def height(self):
        return len(self.reward_map)

    @property
    def width(self):
        return len(self.reward_map[0])

    @property
    def shape(self):
        return self.reward_map.shape

    def actions(self):
        return self.action_space

    def states(self):
        for h in range(self.height):
            for w in range(self.width):
                yield (h, w)
```

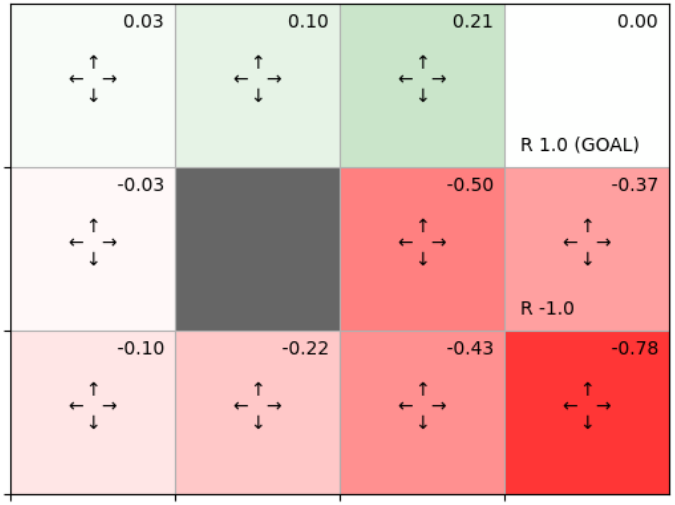
```python
def next_state(self, state, action):
    # 이동 위치 계산
    action_move_map = [(-1, 0), (1, 0), (0, -1), (0, 1)]
    move = action_move_map[action]
    next_state = (state[0] + move[0], state[1] + move[1])
    ny, nx = next_state

    # 이동한 위치가 그리드 월드의 테두리 밖이나 벽인가?
    if nx < 0 or nx >= self.width or ny < 0 or ny >= self.height:
        next_state = state
    elif next_state == self.wall_state:
        next_state = state

    return next_state  # 다음 상태 반환

def reward(self, state, action, next_state):
    return self.reward_map[next_state]

def reset(self):
    self.agent_state = self.start_state
    return self.agent_state

def step(self, action):
    state = self.agent_state
    next_state = self.next_state(state, action)
    reward = self.reward(state, action, next_state)
    done = (next_state == self.goal_state)

    self.agent_state = next_state
    return next_state, reward, done

def render_v(self, v=None, policy=None, print_value=True):
    renderer = render_helper.Renderer(self.reward_map, self.goal_state,
                                      self.wall_state)
    renderer.render_v(v, policy, print_value)

def render_q(self, q=None, print_value=True):
    renderer = render_helper.Renderer(self.reward_map, self.goal_state,
                                      self.wall_state)
    renderer.render_q(q, print_value)
```

# 실습

## 실습 #1 policy_eval.py

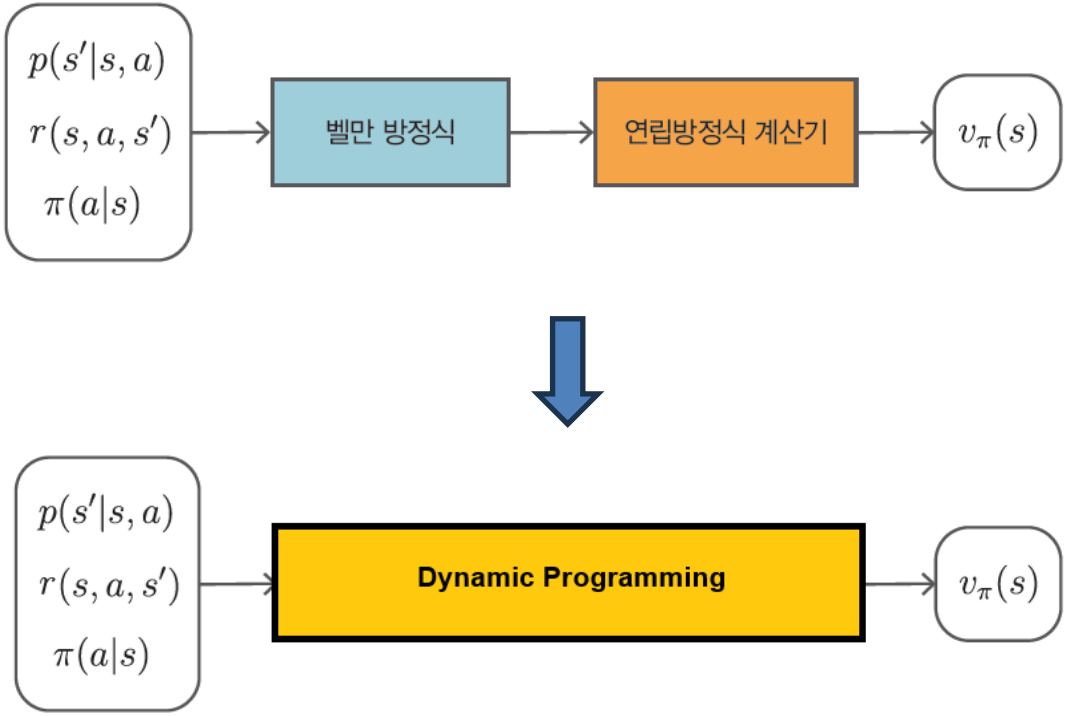
```python
from collections import defaultdict
from gridworld import GridWorld


def eval_onestep(pi, V, env, gamma=0.9):
    for state in env.states():  # 각 상태에 접근
        if state == env.goal_state:  # ❷ 목표 상태에서의 가치 함수는 항상 0
            V[state] = 0
            continue

        action_probs = pi[state]
        new_V = 0

        # 각 행동에 접근
        for action, action_prob in action_probs.items():
            next_state = env.next_state(state, action)
            r = env.reward(state, action, next_state)
            # 새로운 가치 함수
            new_V += action_prob * (r + gamma * V[next_state])

        V[state] = new_V
    return V


def policy_eval(pi, V, env, gamma, threshold=0.001):
    while True:
        old_V = V.copy()  # 갱신 전 가치 함수
        V = eval_onestep(pi, V, env, gamma)

        # 갱신된 양의 최댓값 계산
        delta = 0
        for state in V.keys():
            t = abs(V[state] - old_V[state])
            if delta < t:
                delta = t

        # 임계값과 비교
        if delta < threshold:
            break
    return V
```

```python
if __name__ == '__main__':
    env = GridWorld()
    gamma = 0.9  # 할인율

    pi = defaultdict(lambda: {0: 0.25, 1: 0.25, 2: 0.25, 3: 0.25})  # 정책
    V = defaultdict(lambda: 0)  # 가치 함수

    V = policy_eval(pi, V, env, gamma)  # 정책 평가

    # [그림 4-13] 무작위 정책의 가치 함수
    env.render_v(V, pi)
```



# Policy Evaluation

- 요약



# Bellman Optimality Equation

- Bellman Equation

$$v_\pi(s) = \sum_{a,s'} \pi(a \mid s)\, p(s' \mid s,a)\{r(s,a,s') + \gamma v_\pi(s')\}$$

$$q_\pi(s,a) = \sum_{s'} p(s' \mid s,a)\left\{r(s,a,s') + \gamma \sum_{a'} \pi(a' \mid s') q_\pi(s',a')\right\}$$

- Bellman Optimality Equation

$$v_*(s) = \max_a \sum_{s'} p(s' \mid s,a)\{r(s,a,s') + \gamma v_*(s')\}$$

$$q_*(s,a) = \sum_{s'} p(s' \mid s,a)\{r(s,a,s') + \gamma \max_{a'} q_*(s',a')\}$$

- Optimal Policy

1) stochastic policy $\pi_*(a|s) > 0$ only where $q_*(s,a) = \max_b q_*(s,b)$ (ex) [0.25,0.25,0.25.0.25], [0.5, 0, 0.5, 0]

2) deterministic policy

$$\pi_*(a|s) = \begin{cases} 1 & if\ a = \mu_*(s) \\ 0 & otherwise \end{cases}$$

(ex) [1,0,0,0], [0,0,1,0]

- optimal action

$$\begin{aligned} \mu_*(s) &= \arg\max_a q_*(s,a) \\ &= \arg\max_a \textstyle\sum_{s'} p(s'|s,a)\{r(s,a,s') + \gamma v_\pi(s')\} \\ &= \arg\max_a \{r(s,a,s') + \gamma v_\pi(s')\} \end{aligned}$$

(deterministic state-transition case)

# Policy Improvement

- Policy improvement theorem

Let $\pi$ and $\pi'$ be any pair of deterministic policies such that, for all $s \in \mathcal{S}$,

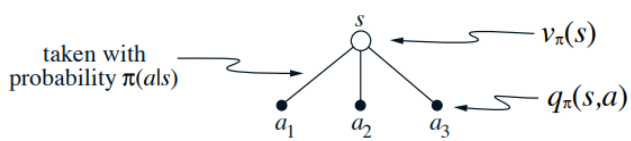
$$q_\pi(s, \mu'(s)) \geq v_\pi(s)$$

then

$$v_{\pi'}(s) \geq v_\pi(s),$$

, where $\pi'(a|s) = \begin{Bmatrix} 1 & if\ a = \mu'(s) \\ 0 & otherwise \end{Bmatrix}$

That is, the policy $\pi'$ must be as good as, or better than $\pi$




$$v_\pi(s) = \sum_a \pi(a \mid s)\, q_\pi(s, a)$$

$$q_\pi(s, a) = \sum_{s'} p(s'|s, a)\{r(s, a, s') + \gamma v_\pi(s')\}$$

- Iteration methods for reaching to the optimal policy

$$\pi_0 \xrightarrow{\text{E}} v_{\pi_0} \xrightarrow{\text{I}} \pi_1 \xrightarrow{\text{E}} v_{\pi_1} \xrightarrow{\text{I}} \pi_2 \xrightarrow{\text{E}} \cdots \xrightarrow{\text{I}} \pi_* \xrightarrow{\text{E}} v_*$$

1) Policy Iteration Method
2) Value Iteration Method

# Policy Iteration Method

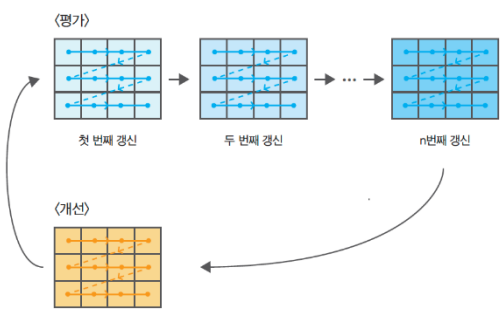
- Policy Iteration Method (정책 반복법)
  - Optimal policy 를 구하기 위한 방법
  - Policy improvement procedure
  - Iterative policy evaluation **and** policy improvement
    - Evaluation: Bellman equation → $v_\pi$
    - Improvement : greedy selection → $\pi$

1. Initialization
   $V(s) \in \mathbb{R}$ and $\pi(s) \in \mathcal{A}(s)$ arbitrarily for all $s \in \mathcal{S}$

2. Policy Evaluation
   Repeat
   $\quad \Delta \leftarrow 0$
   $\quad$ For each $s \in \mathcal{S}$:
   $\quad\quad v \leftarrow V(s)$
   $\quad\quad V(s) \leftarrow \sum_{s',r} p(s', r|s, \pi(s))\left[r + \gamma V(s')\right]$
   $\quad\quad \Delta \leftarrow \max(\Delta, |v - V(s)|)$
   until $\Delta < \theta$ (a small positive number)

3. Policy Improvement
   *policy-stable* $\leftarrow$ *true*
   For each $s \in \mathcal{S}$:
   $\quad a \leftarrow \pi(s)$
   $\quad \pi(s) \leftarrow \arg\max_a \sum_{s',r} p(s', r|s, a)\left[r + \gamma V(s')\right]$
   $\quad$ If $a \neq \pi(s)$, then *policy-stable* $\leftarrow$ *false*
   If *policy-stable*, then stop and return $V$ and $\pi$; else go to 2

$V_0 \rightarrow V_1 \rightarrow \cdots \rightarrow V$

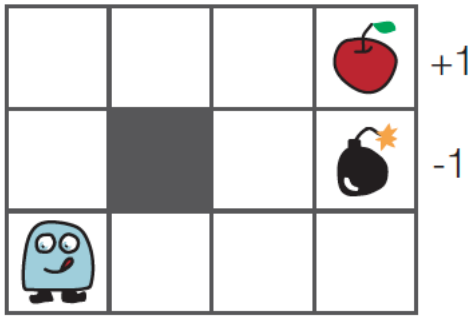



$\pi(s|\mu_*(s)) = 1$

$\mu_*(s) = \arg\max_a \sum_{s'} p(s'|s,a)\{r(s,a,s') + \gamma v_\pi(s')\}$

# Policy Iteration Method

- Example: 3 x 4 Grid World




Deterministic Policy

$$\mu_*(s) = \arg\max_a q_*(s,a)$$
$$= \arg\max_a \sum_{s'} p(s'|s,a)\{r(s,a,s') + \gamma v_\pi(s')\}$$

Deterministic state-transition

$$\mu_*(s) = \arg\max_a \{r(s,a,s') + \gamma v_\pi(s')\}$$

# Policy Iteration Method

- argmax()

```
def argmax(d):
    max_value = max(d.values())
    max_key = 0
    for key, value in d.items():
        if value == max_value:
            max_key = key
    return max_key
```

```
action_values = {0: 0.1, 1: -0.3, 2: 9.9, 3: -1.3}  # 9.9가 최댓값(키는 2)

max_action = argmax(action_values)
print(max_action)
```

```
2
```

# Policy Iteration Method

- greedy_policy()

```
def greedy_policy(V, env, gamma):
    pi = {}

    for state in env.states():
        action_values = {}

        for action in env.actions():
            next_state = env.next_state(state, action)
            r = env.reward(state, action, next_state)
            value = r + gamma * V[next_state]  # ❶
            action_values[action] = value

        max_action = argmax(action_values)  # ❷
        action_probs = {0: 0, 1: 0, 2: 0, 3: 0}
        action_probs[max_action] = 1.0
        pi[state] = action_probs  # ❸
    return pi
```

$q(s,a) = r(s,a,s') + \gamma v_\pi(s)$

$\mu_*(s) = \arg\max_a q(s,a)$

$\pi_*(a|s)$

# Policy Iteration Method

- policy_iter()

```
def policy_iter(env, gamma, threshold=0.001, is_render=False):
    pi = defaultdict(lambda: {0: 0.25, 1: 0.25, 2: 0.25, 3: 0.25})
    V = defaultdict(lambda: 0)

    while True:
        V = policy_eval(pi, V, env, gamma, threshold)  # ❶ 평가
        new_pi = greedy_policy(V, env, gamma)          # ❷ 개선

        if is_render:
            env.render_v(V, pi)

        if new_pi == pi:  # ❸ 갱신 여부 확인
            break
        pi = new_pi

    return pi
```

Policy evaluation

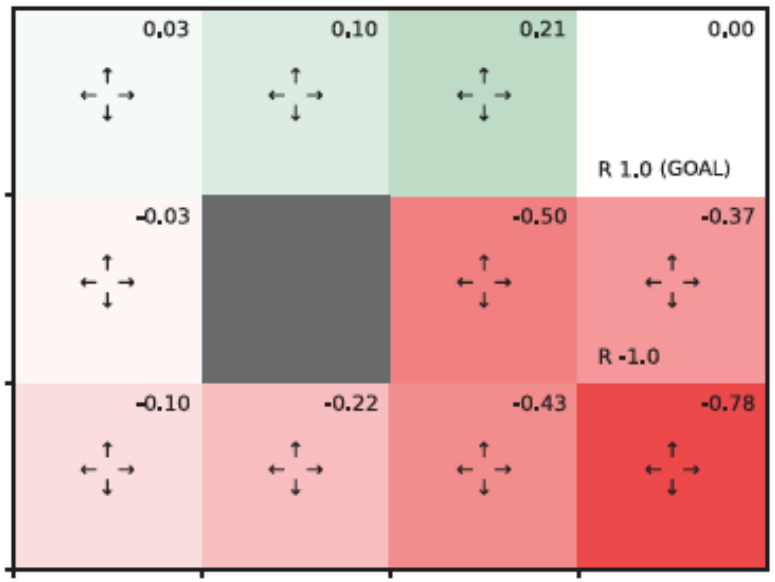
Policy improvement (update)

$\pi_*(a|s)$

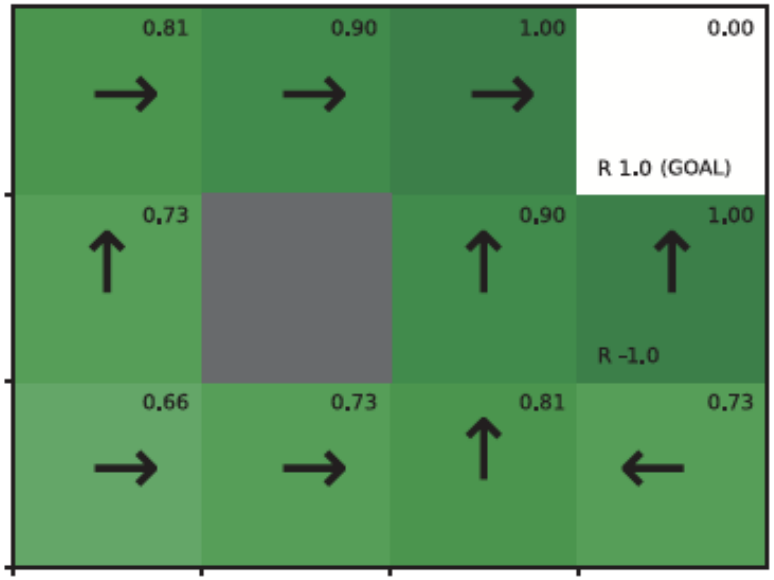
# Policy Iteration Method

- $\mu_*(s)$ & $v_*(s)$

```
env = GridWorld()
gamma = 0.9
pi = policy_iter(env, gamma)
```




처음




마지막(네 번째 갱신)

# 실습

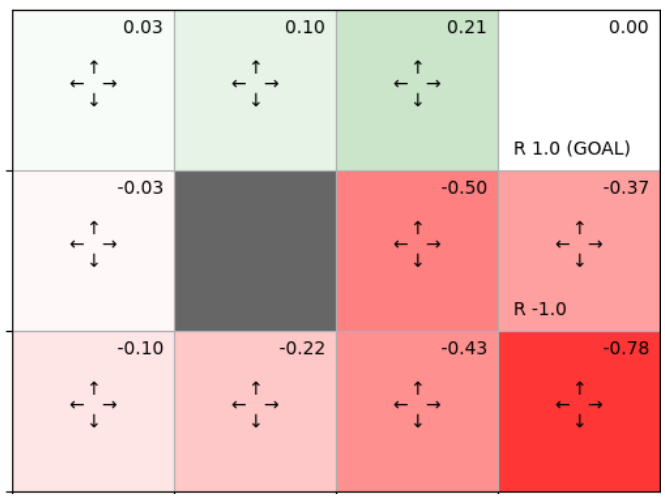
## 실습 #2 policy_iter.py

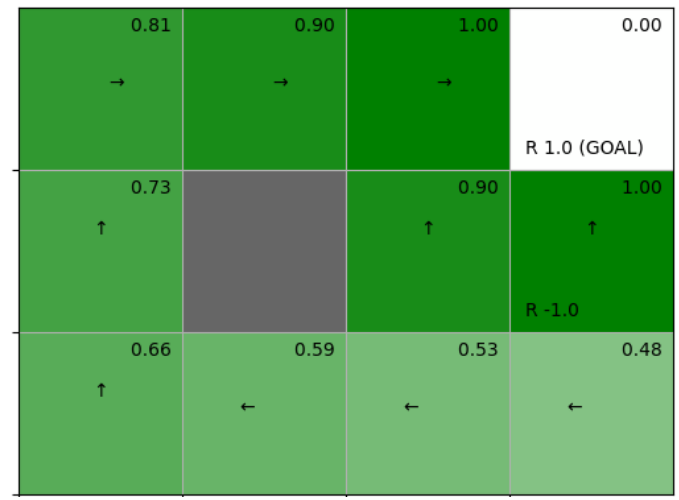
```python
from collections import defaultdict
from common.gridworld import GridWorld
from policy_eval import policy_eval


def argmax(d):
    """d (dict)"""
    max_value = max(d.values())
    max_key = -1
    for key, value in d.items():
        if value == max_value:
            max_key = key
    return max_key


def greedy_policy(V, env, gamma):
    pi = {}

    for state in env.states():
        action_values = {}

        for action in env.actions():
            next_state = env.next_state(state, action)
            r = env.reward(state, action, next_state)
            value = r + gamma * V[next_state]
            action_values[action] = value

        max_action = argmax(action_values)
        action_probs = {0: 0, 1: 0, 2: 0, 3: 0}
        action_probs[max_action] = 1.0
        pi[state] = action_probs
    return pi
```

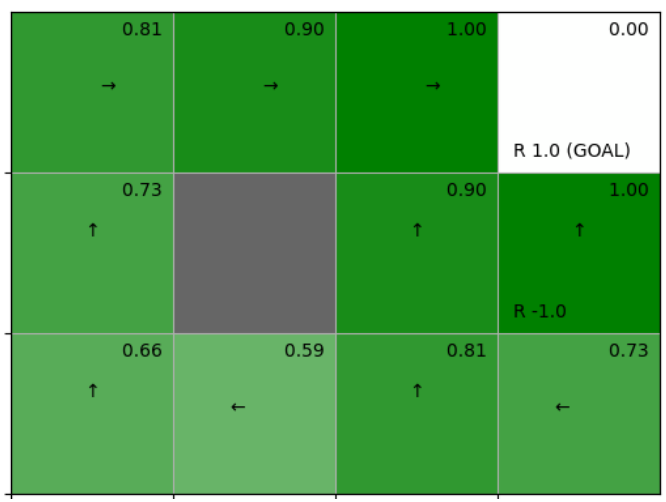
```python
def policy_iter(env, gamma, threshold=0.001, is_render=True):
    pi = defaultdict(lambda: {0: 0.25, 1: 0.25, 2: 0.25, 3: 0.25})
    V = defaultdict(lambda: 0)

    while True:
        V = policy_eval(pi, V, env, gamma, threshold)  # 평가
        new_pi = greedy_policy(V, env, gamma)          # 개선

        if is_render:
            env.render_v(V, pi)

        if new_pi == pi:  # 갱신 여부 확인
            break
        pi = new_pi

    return pi


# 가치 함수와 정책 시각화(책의 [그림 4-16]은 처음과 마지막 그래프만 표시했음)
if __name__ == '__main__':
    env = GridWorld()
    gamma = 0.9
    pi = policy_iter(env, gamma)
```

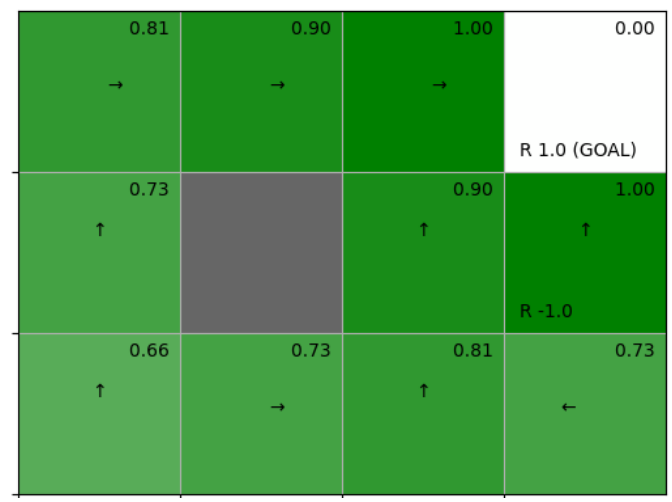
# 실습

## 실습 #2


0.03
0.10
0.21
0.00
R 1.0 (GOAL)
-0.03
-0.50
-0.37
R -1.0
-0.10
-0.22
-0.43
-0.78


0.81
0.90
1.00
0.00
R 1.0 (GOAL)
0.73
0.90
1.00
R -1.0
0.66
0.59
0.53
0.48


0.81
0.90
1.00
0.00
R 1.0 (GOAL)
0.73
0.90
1.00
R -1.0
0.66
0.59
0.81
0.73


0.81
0.90
1.00
0.00
R 1.0 (GOAL)
0.73
0.90
1.00
R -1.0
0.66
0.73
0.81
0.73

# Value Iteration Method

- Value Iteration Method (가치 반복법)
  - Optimal policy 를 구하기 위한 방법
  - Policy improvement procedure
  - Iterative policy evaluation **with** improvement
    - Evaluation: Bellman Optimality Equation → $v_\pi$
    - Improvement : greedy selection → $\pi$

<특징>

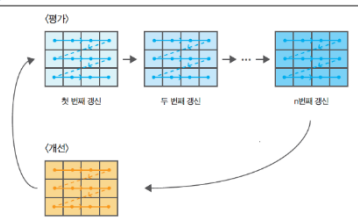
- 불필요한 계산 제거
- 일반적으로 수렴속도가 빠르다

Policy Iteration Method

1. Initialization
   $V(s) \in \mathbb{R}$ and $\pi(s) \in \mathcal{A}(s)$ arbitrarily for all $s \in \mathcal{S}$

2. Policy Evaluation
   Repeat
   $\quad \Delta \leftarrow 0$
   $\quad$ For each $s \in \mathcal{S}$:
   $\quad\quad v \leftarrow V(s)$
   $\quad\quad V(s) \leftarrow \sum_{s',r} p(s',r|s,\pi(s))\left[r + \gamma V(s')\right]$
   $\quad\quad \Delta \leftarrow \max(\Delta, |v - V(s)|)$
   until $\Delta < \theta$ (a small positive number)

3. Policy Improvement
   *policy-stable* $\leftarrow$ *true*
   For each $s \in \mathcal{S}$:
   $\quad a \leftarrow \pi(s)$
   $\quad \pi(s) \leftarrow \arg\max_a \sum_{s',r} p(s',r|s,a)\left[r + \gamma V(s')\right]$
   $\quad$ If $a \neq \pi(s)$, then *policy-stable* $\leftarrow$ *false*
   If *policy-stable*, then stop and return $V$ and $\pi$; else go to 2

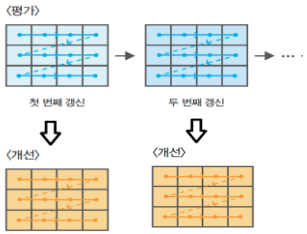



Value Iteration Method

Initialize array $V$ arbitrarily (e.g., $V(s) = 0$ for all $s \in \mathcal{S}^+$)

Repeat
$\quad \Delta \leftarrow 0$
$\quad$ For each $s \in \mathcal{S}$:
$\quad\quad v \leftarrow V(s)$
$\quad\quad V(s) \leftarrow \max_a \sum_{s',r} p(s',r|s,a)\left[r + \gamma V(s')\right]$
$\quad\quad \Delta \leftarrow \max(\Delta, |v - V(s)|)$
until $\Delta < \theta$ (a small positive number)

Output a deterministic policy, $\pi$, such that
$\quad \pi(s) = \arg\max_a \sum_{s',r} p(s',r|s,a)\left[r + \gamma V(s')\right]$



# Value Iteration Method

- Evaluation Equation
  - Policy iteration method

$$v_\pi(s) = \sum_{a,s'} \pi(a \mid s)\, p(s' \mid s,a)\{r(s,a,s') + \gamma v_\pi(s')\}$$

Bellman equation

$$V'(s) = \sum_{a,s'} \pi(a \mid s)\, p(s' \mid s,a)\{r(s,a,s') + \gamma V(s')\}$$

계산 시 policy $\pi(a|s)$ or $\mu(s)$ 필요

  - Value iteration method

$$v_*(s) = \max_a \sum_{s'} p(s' \mid s,a)\{r(s,a,s') + \gamma v_*(s')\}$$

Bellman optimality equation

$$V'(s) = \max_a \sum_{s'} p(s' \mid s,a)\{r(s,a,s') + \gamma V(s')\}$$

계산 시 policy $\pi(a|s)$ or $\mu(s)$ 불필요

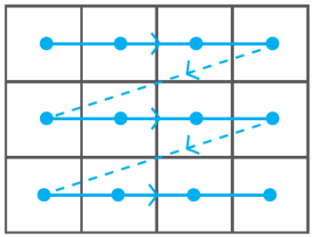
- Improvement Equation

$$\mu(s) = \arg\max_a \sum_{s'} p(s' \mid s,a)\{r(s,a,s') + \gamma V(s')\}$$

# Value Iteration Method

- value_iter_onestep(): $V_k \rightarrow V_{k+1}$

```
def value_iter_onestep(V, env, gamma):
    for state in env.states():        # ❶ 모든 상태에 차례로 접근
        if state == env.goal_state:   # 목표 상태에서의 가치 함수는 항상 0
            V[state] = 0
            continue

        action_values = []
        for action in env.actions():  # ❷ 모든 행동에 차례로 접근
            next_state = env.next_state(state, action)
            r = env.reward(state, action, next_state)
            value = r + gamma * V[next_state]  # ❸ 새로운 가치 함수
            action_values.append(value)

        V[state] = max(action_values)  # ❹ 최댓값 추출
    return V
```



$s' = f(s, a))$

$r(s, a, s') + \gamma V(s')$

$V'(s) = \max_a \{r(s, a, s') + \gamma V'(s')\}$

# Value Iteration Method

- value_iter() : $V_0 \rightarrow V_1 \rightarrow \cdots$

```
def value_iter(V, env, gamma, threshold=0.001, is_render=True):
    while True:
        if is_render:
            env.render_v(V)

        old_V = V.copy()  # 갱신 전 가치 함수
        V = value_iter_onestep(V, env, gamma)

        # 갱신된 양의 최댓값 구하기
        delta = 0
        for state in V.keys():
            t = abs(V[state] - old_V[state])
            if delta < t:
                delta = t

        # 임곗값과 비교
        if delta < threshold:
            break
    return V
```

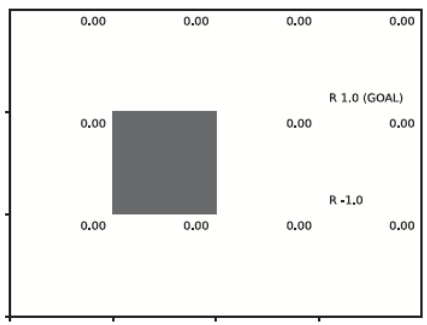
$V_k'(s)$

$k = 1,2,\cdots,$

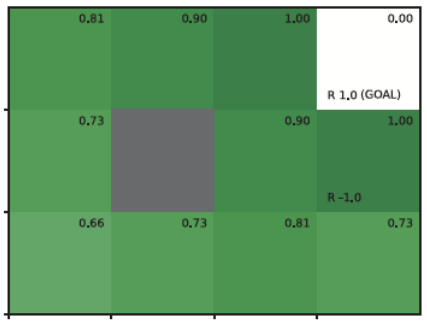
# Value Iteration Method

- Optimal policy

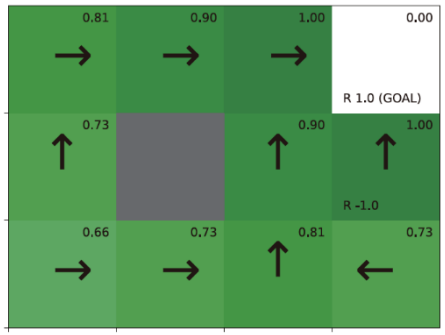
```
V = defaultdict(lambda: 0)
env = GridWorld()
gamma = 0.9

V = value_iter(V, env, gamma)  # 최적 가치 함수 찾기

pi = greedy_policy(V, env, gamma)  # 최적 정책 찾기
env.render_v(V, pi)
```




처음



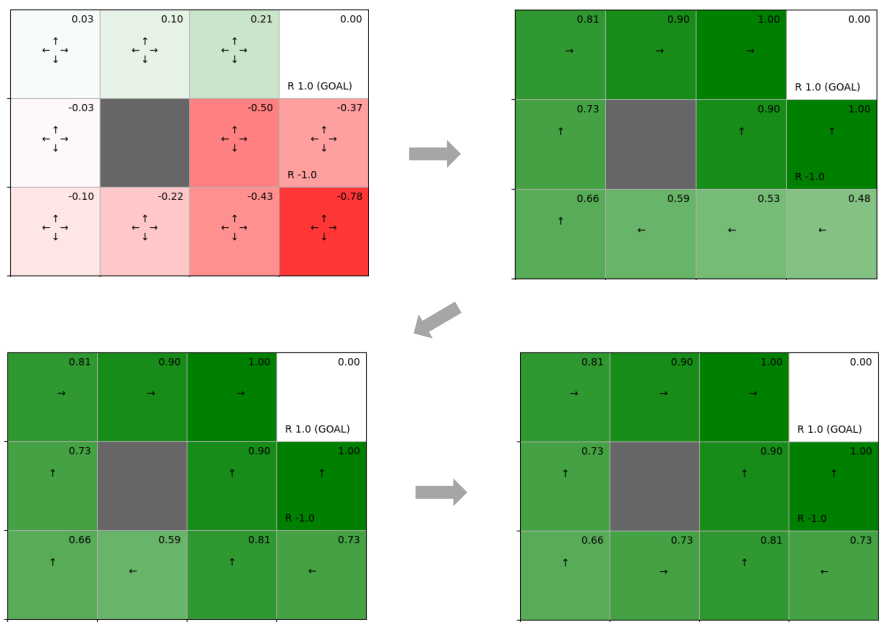

마지막(세 번째 갱신)



# Policy Iteration vs. Value Iteration

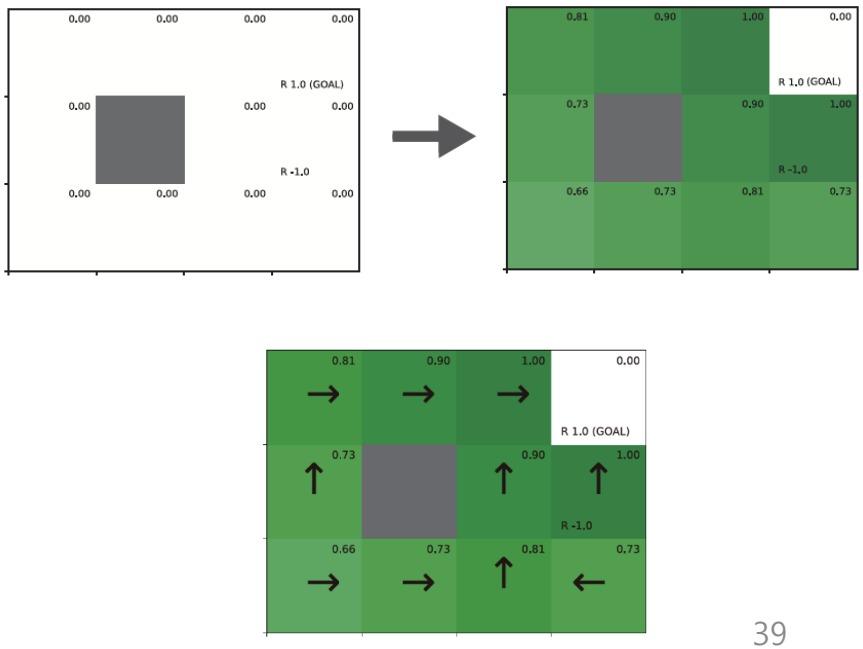
## Policy Iteration Method

```
pi = defaultdict(lambda: {0: 0.25, 1: 0.25, 2: 0.25, 3: 0.25})
V = defaultdict(lambda: 0)

  while True:
      V = policy_eval(pi, V, env, gamma, threshold)  # 평가
      new_pi = greedy_policy(V, env, gamma)          # 개선

      if new_pi == pi:  # 갱신 여부 확인
          break
      pi = new_pi

  return pi
```




## Value Iteration Method

```
V = defaultdict(lambda: 0)
env = GridWorld()
gamma = 0.9

V = value_iter(V, env, gamma)       # 최적가치함수
pi = greedy_policy(V, env, gamma)   # 최적정책
env.render_v(V, pi)
```



# 실습

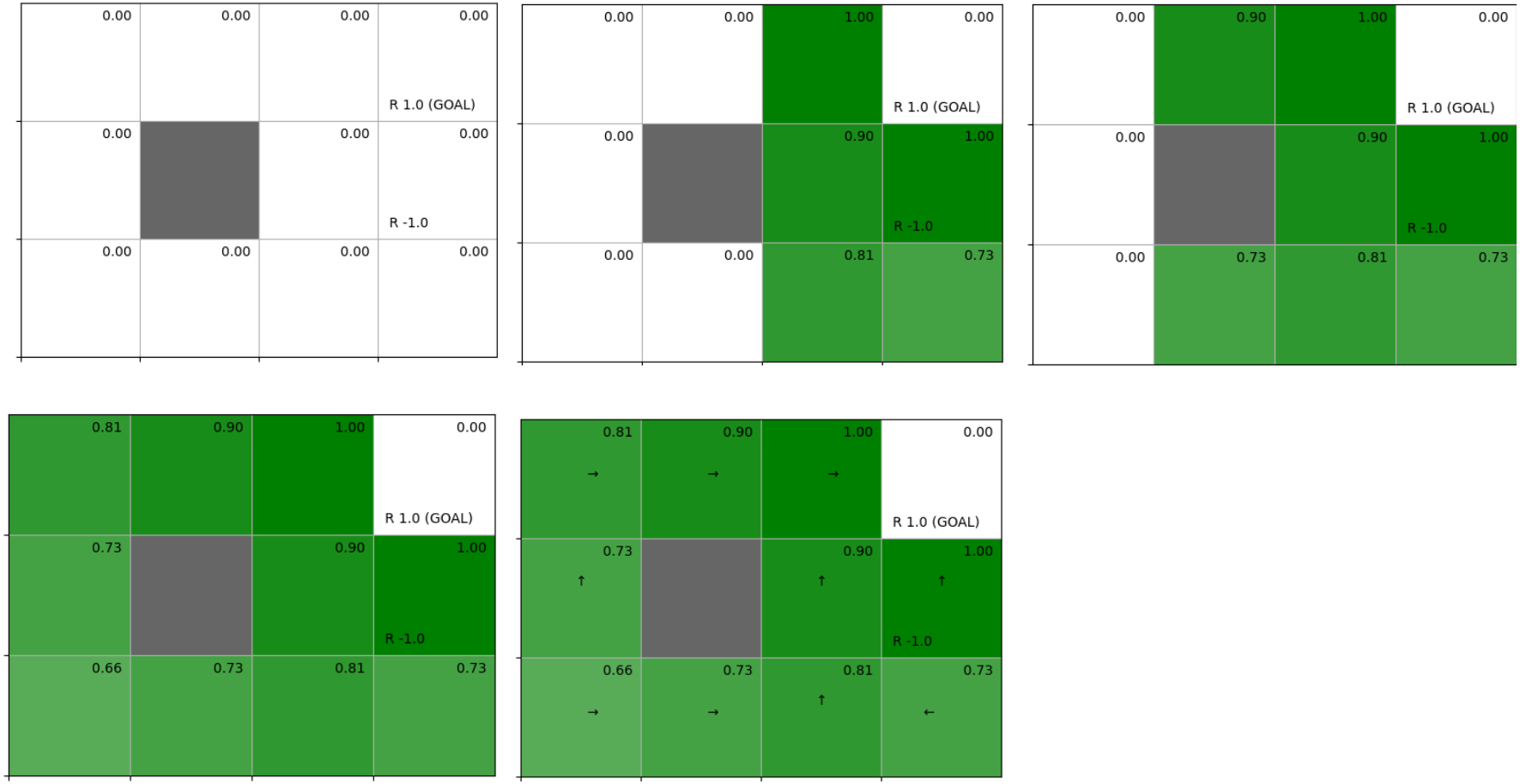
실습 #3 value_iter.py

```python
from collections import defaultdict
from common.gridworld import GridWorld
from policy_iter import greedy_policy


def value_iter_onestep(V, env, gamma):
    for state in env.states():        # 모든 상태에 차례로 접근
        if state == env.goal_state:  # 목표 상태에서의 가치 함수는 항상 0
            V[state] = 0
            continue

        action_values = []
        for action in env.actions():  # 모든 행동에 차례로 접근
            next_state = env.next_state(state, action)
            r = env.reward(state, action, next_state)
            value = r + gamma * V[next_state]  # 새로운 가치 함수
            action_values.append(value)

        V[state] = max(action_values)  # 최댓값 추출
    return V
```

```python
def value_iter(V, env, gamma, threshold=0.001, is_render=True):
    while True:
        if is_render:
            env.render_v(V)

        old_V = V.copy()  # 갱신 전 가치 함수
        V = value_iter_onestep(V, env, gamma)

        # 갱신된 양의 최댓값 구하기
        delta = 0
        for state in V.keys():
            t = abs(V[state] - old_V[state])
            if delta < t:
                delta = t

        # 임계값과 비교
        if delta < threshold:
            break
    return V


if __name__ == '__main__':
    V = defaultdict(lambda: 0)
    env = GridWorld()
    gamma = 0.9

    V = value_iter(V, env, gamma)  # 최적 가치 함수 찾기

    pi = greedy_policy(V, env, gamma)  # 최적 정책 찾기
    env.render_v(V, pi)
```

# 실습

## 실습 #3

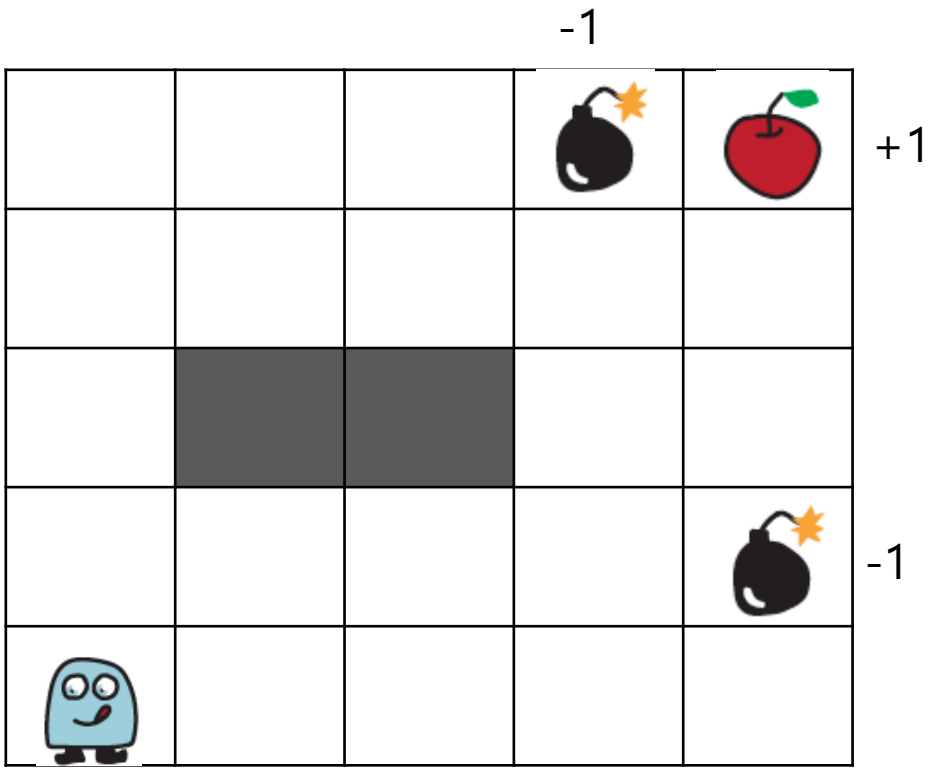

0.00 0.00 0.00 0.00
R 1.0 (GOAL)
0.00 0.00 0.00
R -1.0
0.00 0.00 0.00 0.00
0.00 0.00 1.00 0.00
R 1.0 (GOAL)
0.00 0.90 1.00
R -1.0
0.00 0.00 0.81 0.73
0.00 0.90 1.00 0.00
R 1.0 (GOAL)
0.00 0.90 1.00
R -1.0
0.00 0.73 0.81 0.73
0.81 0.90 1.00 0.00
R 1.0 (GOAL)
0.73 0.90 1.00
R -1.0
0.66 0.73 0.81 0.73
0.81 0.90 1.00 0.00
→ → →
R 1.0 (GOAL)
0.73 0.90 1.00
↑ ↑ ↑
R -1.0
0.66 0.73 0.81 0.73
→ → ↑ ←

# Quiz

(Q) 다음의 5x5 Grid World 에 대한 value function 및 policy 를 구하라.

(1)Policy iteration method

(2)Value iteration method



# 요약

- Optimal Policy 를 어떻게 구하는가?
  - Policy improvement procedure

$$\pi_0 \xrightarrow{\text{E}} v_{\pi_0} \xrightarrow{\text{I}} \pi_1 \xrightarrow{\text{E}} v_{\pi_1} \xrightarrow{\text{I}} \pi_2 \xrightarrow{\text{E}} \cdots \xrightarrow{\text{I}} \pi_* \xrightarrow{\text{E}} v_*$$

    1) policy iteration method: evaluation and improvement
    2) Value iteration method: evaluation with improvement

  - Policy evaluation ⇒ dynamic programming
    1) policy iteration method: Bellman equation
    2) Value iteration method: Bellman optimality equation

  - Policy improvement ⇒ greedy selection

- Problem
  - Dynamic programming
    - Curse of dimensionality
    - Needs the model of environment: $p(s'|s,a)$, $r(s,a,s')$